

Ensemble Learning (Bagging and Boosting)

Motivation

- Ensembling → Learning from more than one classifier
- A single classifier may not be powerful enough
 - Limited hypothesis class: example linear classifier, or decision tree of limited depth
- A single classifier may overfit
 - Decision tree without no limit on length
 - Neural network without regularizer
- Many competitions won because of ensembling!
- Ensembling continues to be useful even in the era of deep learning

Bias and Variance of a model class

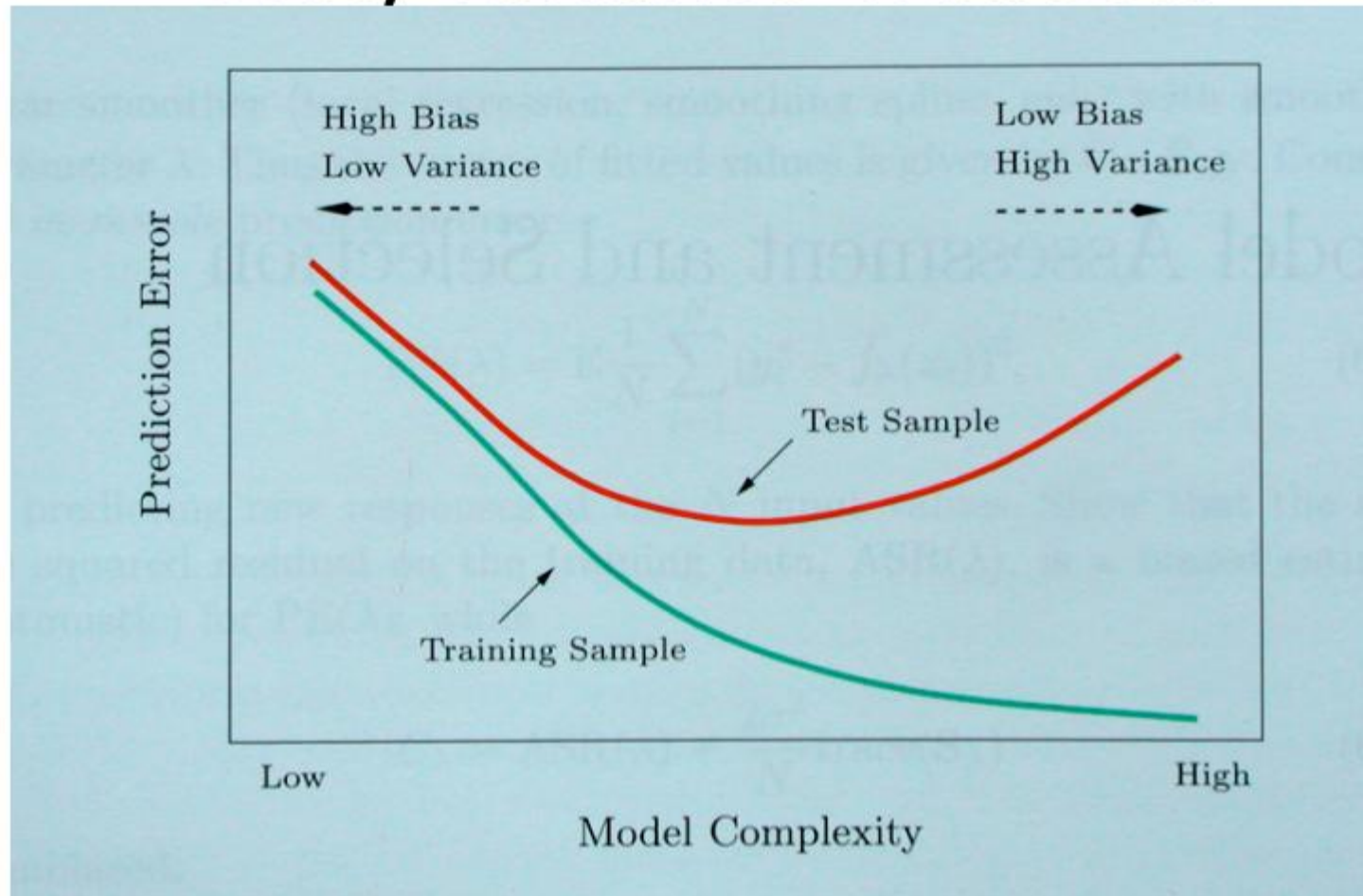
- Bias:

- Simplifying assumptions about the form of the model so as to be easy to learn.
- Example: linear regression can only learn linear separators, naïve Bayes assumes conditional independence
- Low bias classifiers: Kernel SVMs, nearest neighbor classifier, feed forward NN, decision trees,
- High bias classifiers: linear regression, naïve Bayes, perceptron, in general any parameteric model has high bias.

- Variance:

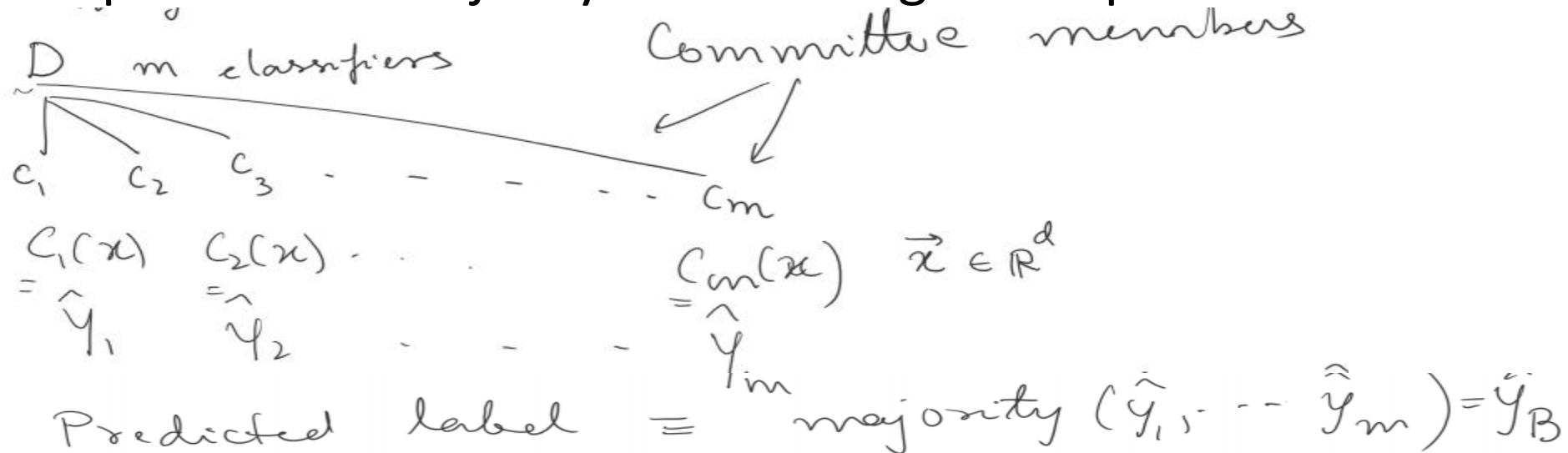
- Change in accuracy of the model with changes in the training dataset.
- For a high variance classifier the test accuracy will change a lot when we sample different N instances for training from the data distribution.
- Examples of high variance classifier: Kernel SVMs, nearest neighbor classifier, feed forward NN with many units, unbounded depth decision tree
- Examples of low variance classifier: linear regression, naïve Bayes, perceptron, small depth decision trees.

Bias variance tradeoff



Bagging

- Accuracy from a single classifier may have high variance
- Bagging creates a committee of classifiers and averages the predictions from the committee to make the final prediction
- D = training set, m = number of classifiers or committee members
- Train M classifiers: $C_1(x)$, $C_2(x)$, ... $C_m(x)$
- For a test instance, get m predictions $\hat{y}_1, \dots, \hat{y}_m$
- Final prediction: majority label among the m predictions



Why should bagging reduce error?

- Simple Example (Synthetic)

* Assume binary classification
* Assume each classifier is correct with probability " p " = $P(C_j(x) = y)$

* Assume that the correctness of a classifier C_j is independent of the " " another C_i

$$P(C_i(x) = y, \wedge C_j(x) = y) = P(C_i(x) = y) P(C_j(x) = y)$$

* $\hat{y}_B = \text{majority}(C_1(x), \dots, C_m(x))$

* What is probability that $\hat{y}_B$ is correct?

$$= \sum_{k=\frac{m}{2}+1}^m \binom{m}{k} p^k (1-p)^{m-k} \geq p \text{ when } p \geq 0.5$$

m is odd

HW: generalize to multi-class.

Methods of creating committee

Two goals during creation of committee members: each classifier should be as accurate as possible, the predictions from different classifiers should be as independent as possible.

- Bootstrap sampling
 - Create different training samples from the given training dataset D

$D_1, D_2 \dots D_M$

- Random forests
 - Above + Create different random attribute subsets from D

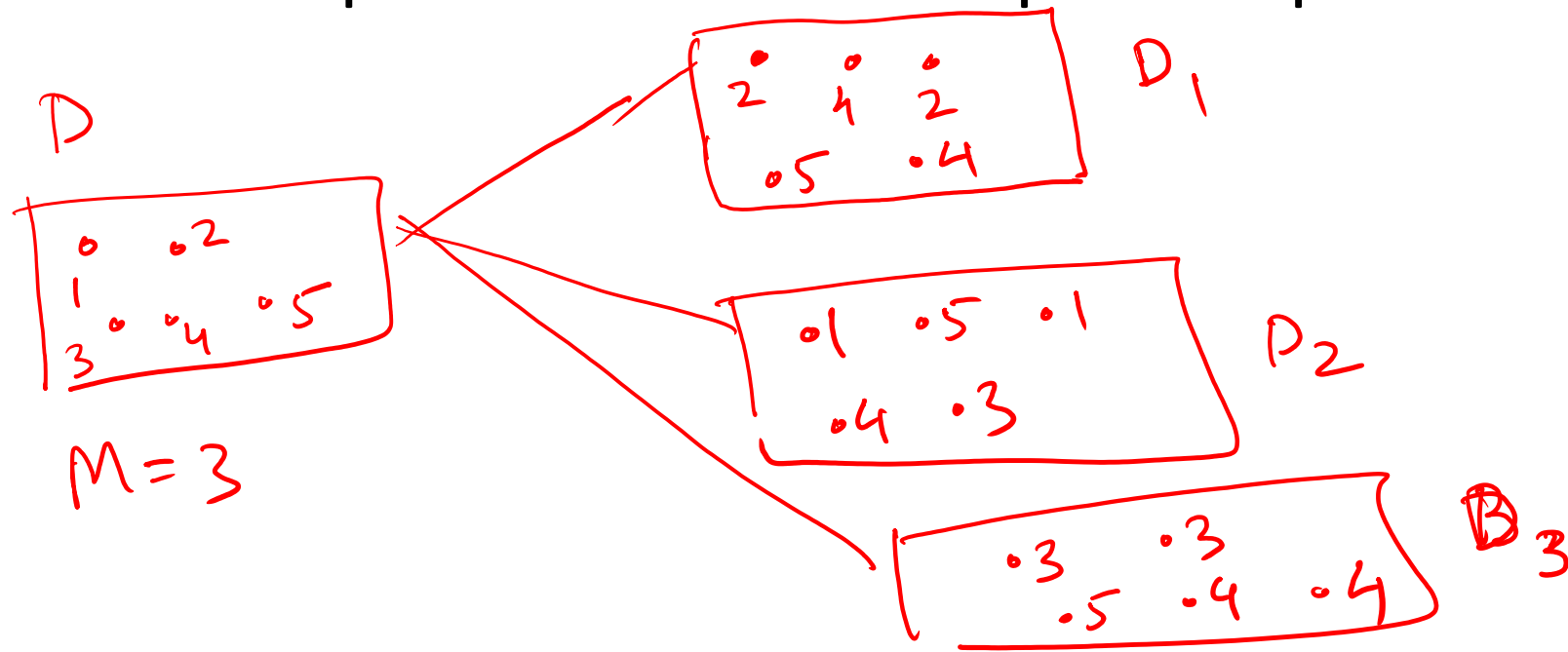
Use D_j and randomized attribute selection to train a tree C_j

Bagging by Bootstrap sampling

$$\begin{aligned} & \overline{D = \{ (x^1, y^1) \dots (x^N, y^N) \}} \\ & \downarrow \text{Convert this into a point distribution} \\ & \underline{\underline{P_D(x, y) = \frac{1}{N} \text{ if } (x, y) \in D}} \\ & \quad = 0 \text{ otherwise.} \end{aligned}$$

- For $j = 1$ to M
 - /* Create training set for D_j using bootstrap sampling as follows */
 - For $i=1$ to N
 - Sample an instance from D (uniformly from P_D) /* Sampling with replacement */
 - $C_j(x) =$ Train j -th committee member using training data D_j

Example of bootstrap samples for bagging



what is the expected # of distinct samples in any one bootstrapped dataset D_j

$$N \left(1 - \left(1 - \frac{1}{N} \right)^N \right)$$

at $N \rightarrow \infty$ $\left(1 - \frac{1}{N} \right)^N = \frac{1}{e}$

$\left(1 - \frac{1}{e} \right) \approx 0.632$

Expected number of distinct samples in any one bag after bootstrap sampling?

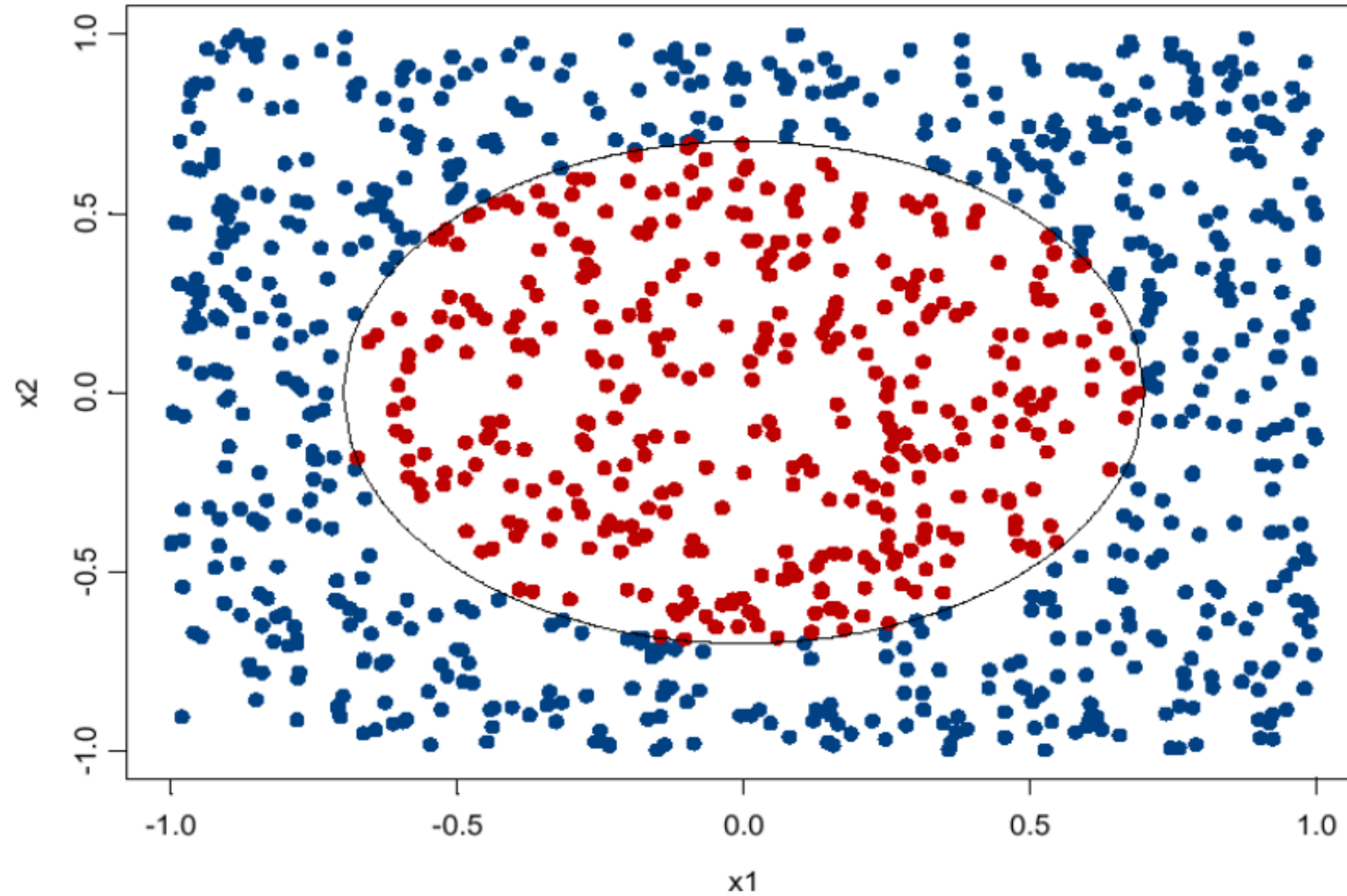
N *Probability that an instance is selected in any of N trials

$$N \left(1 - \left(1 - \frac{1}{N} \right)^N \right) \longrightarrow N \left(1 - \frac{1}{e} \right) \approx 0.63N$$

Bagging

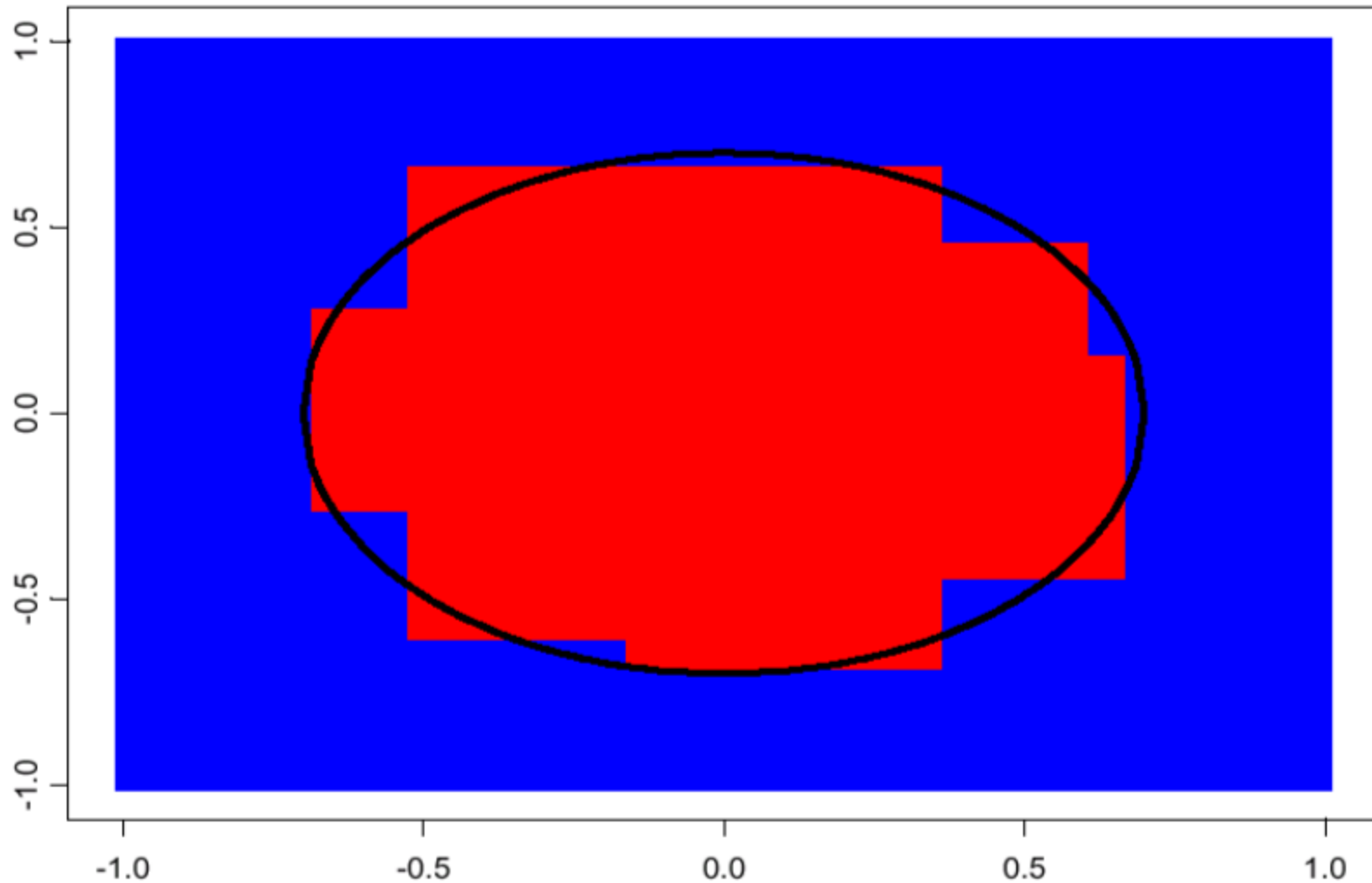
- Useful when individual classifiers are over-fitting, e.g. a decision tree without pruning.
- Bagging by averaging the predictions from multiple over-fitted trees reduces variance

Bagging Example

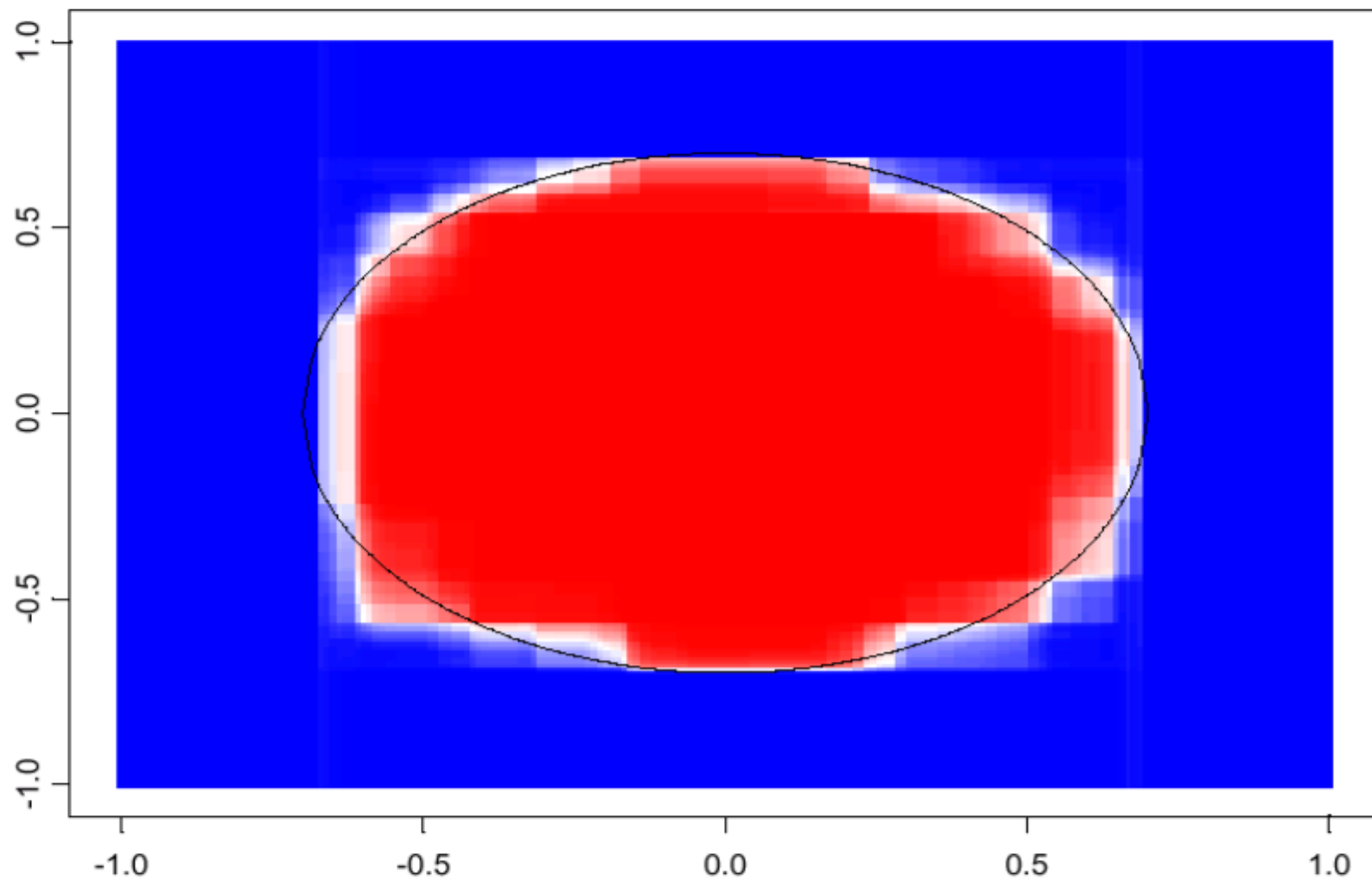


decision tree learning algorithm; very similar to ID3

CART decision boundary



100 bagged trees



shades of blue/red indicate strength of vote for particular classification

Random Forests

- Create m decision trees.
- Each tree uses a bootstrap sample from D
- For each node of each tree
 - Randomly sample $\sqrt{d}$ attributes from the available d attributes
 - Select the best split from among the sampled attributes

Many other variants of randomize selection of attributes [skipping those]

Random forests one of the best performing of the traditional classifiers in many applications.

Random forests Algorithm

1. For $b = 1$ to B :
 - (a) Draw a **bootstrap sample** Z_b^* of size N from the training data.
 - (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
 - i. Select **m variables at random** from the p variables.
 - ii. Pick the best variable/split-point among the m .
 - iii. Split the node into two daughter nodes.
2. Output the ensemble of trees $\{T_b\}_1^B$.

To make a prediction at a new point x :

Regression: $\hat{f}_{\text{rf}}^B(x) = \frac{1}{B} \sum_{b=1}^B T_b(x).$

Classification: Let $\hat{C}_b(x)$ be the class prediction of the b th random-forest tree. Then $\hat{C}_{\text{rf}}^B(x) = \text{majority vote } \{\hat{C}_b(x)\}_1^B.$

Other methods of Ensembling

- Bagging by varying random factors during model creation
 - Different initial values of parameters (Non-convex models like NN)
 - Bayesian models create a posterior distribution over parameters and different committee members obtained by sampling from the posterior
- Boosting
 - Create classifiers sequentially where each subsequent classifier focuses more on examples misclassified by previous classifiers.
 - Focus handled via weights attached with each example
 - Can potentially increase bias
- Stacking
- Weighted Bagging

Boosting (Chapter 18 of Prob ML book)

- Model's form is weighted sum of component models, like in bagging, but the training method is different
- Form of model:
 - $f(x; \theta) = \sum_1^M \beta_m F_m(x, \theta_m)$
- Sequentially train weak learners where subsequent learners focus on residues of the previous stages → complicated decision boundaries from simple classifiers
- Simple classifier examples:
 - Naïve Bayes classifier
 - Decision stumps
 - Linear classifiers.

Forward stagewise additive boosting

- Sequentially optimize the overall training loss while keeping previous stages fixed.
- Given training data and loss:

$$D = \{(x^1, y^1), \dots, (x^N, y^N)\}$$
$$\text{loss-function: } \sum_{i=1}^N l(y_i, \underline{f(x^i)}) = \sum_{i=1}^N l(y_i, \sum_{m=1}^M \beta_m F_m(x_i))$$

- Optimize the loss function iteratively:

$$\text{Initially: } \vec{\theta}_1 \leftarrow \underset{\vec{\theta}}{\operatorname{argmin}} \sum_{i=1}^N l(y_i, \beta_1(F_1(x_i, \vec{\theta})))$$

$$\text{for } m = 1 \text{ to } M$$
$$\vec{\theta}_m \leftarrow \underset{\vec{\theta}}{\operatorname{argmin}} \sum_{i=1}^N l(y_i, \underline{f_{m-1}(x_i) + \beta_m F_m(x_i, \vec{\theta})})$$
$$f_m(x^i) = \sum_{j=1}^m f_j(x_i)$$

Boosted regression trees with square loss

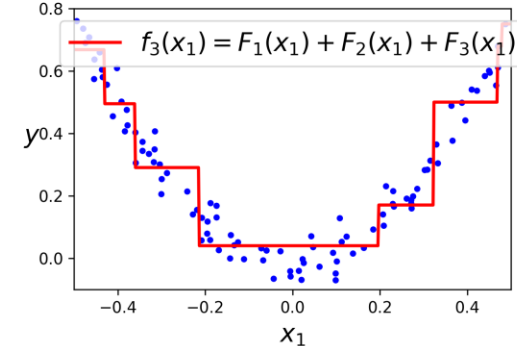
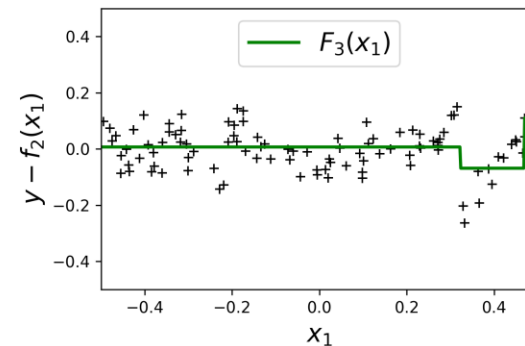
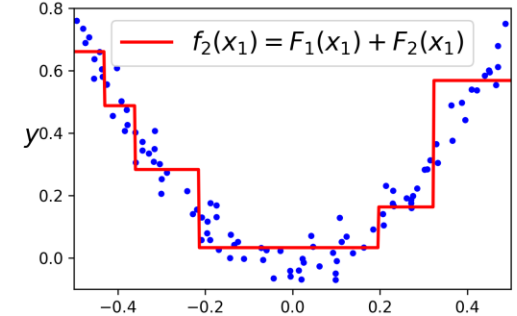
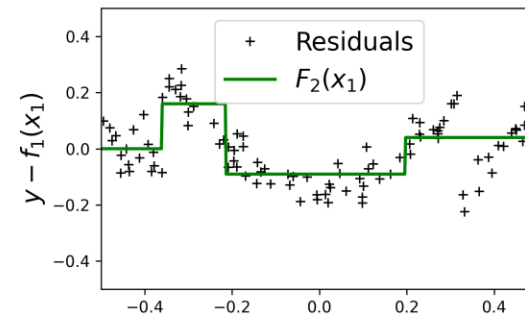
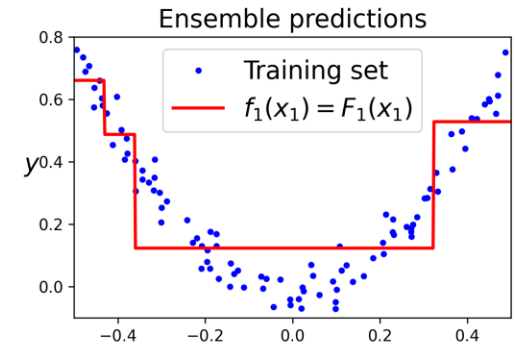
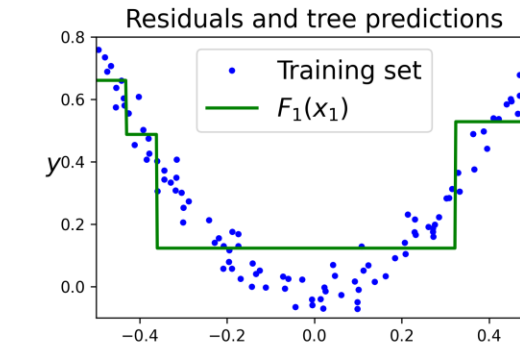
When loss function is square loss

*m*th step

$$\min_{\theta} \sum_{i=1}^N \left(y_i - \underbrace{f_{m-1}(x_i)} - \beta_m F_m(x_i; \theta) \right)^2$$
$$\min_{\theta} \sum_{i=1}^N \left(y_{im} - \beta_m F_m(x_i; \theta) \right)^2$$

Assume $\beta_m = 1$, the above reduces to standard least square regression.

- <https://drive.google.com/file/d/1a7kfsrBtjwhP4x7XbwOtbWPG3oxdP5WS/view?usp=sharing>



Adaboost

- Boosting with exponential loss.
- A principled method that solves for both $\beta_m, F_m(x)$ in closed form for exponential loss.
 - Will cover if time permits.
 - Could not cover. Not in syllabus.
- Limitations:
 - Requires classification methods to be modified to work with weighted examples.
 - Exponential loss does not work well on noisy real-life data
- In practice the next method we discuss, Gradient boosting much better.

Gradient boosting algorithm (Chapter 18.5.5 of PML book)

- A general-purpose algorithm for any differentiable loss function.
- Given N training examples, view $f_i = f(x^i)$ as an independent parameter

$$f_i = \sum_{m=1}^M \beta_m f_m(x^i, \theta_m)$$

Initially $f_i = F_1(x^i, \theta_1)$

- At each stage m , we have a current value of the function.

$$f_{i,m-1} = \sum_{j=1}^{m-1} \beta_j F_j(x^i, \theta_j) \rightarrow f_{i,m+1}$$

- Find gradient of the loss with respect to each f_i independently.

$$r_{im} = \frac{\partial \ell(y_i, f_i)}{\partial f_i} \Big|_{f_i = f_{i,m-1}} \Rightarrow \overset{\text{Apply gradient}}{f_{i,m} = f_{i,m-1} + \alpha_m r_{im}}$$

X You do not generalize.

Continued

• ~~A~~

$$4) \quad \theta_m \leftarrow \underset{\theta}{\operatorname{argmin}} \left(x_{im} - F_m(x^i) \right)^2$$

$$5) \quad \beta_m \leftarrow \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^N \ell \left(y_i, \underbrace{f_{m-1}(x^i) - \beta x_{im}} \right)$$

- Function at the m th stage F_m is trained to match the gradients.

Algorithm 18.3: Gradient boosting

```
1 Initialize  $f_0(\mathbf{x}) = \operatorname{argmin}_F \sum_{i=1}^N L(y_i, F(\mathbf{x}_i))$ 
2 for  $m = 1 : M$  do
3   Compute the gradient residual using  $r_{im} = - \left[ \frac{\partial L(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)} \right]_{f(\mathbf{x}_i)=f_{m-1}(\mathbf{x}_i)}$ 
4   Use the weak learner to compute  $F_m = \operatorname{argmin}_F \sum_{i=1}^N (r_{im} - F(\mathbf{x}_i))^2$ 
5   Update  $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \underset{\mathcal{P}}{\nu} F_m(\mathbf{x})$ 
6 Return  $f(\mathbf{x}) = f_M(\mathbf{x})$ 
```

Example: gradient for square loss

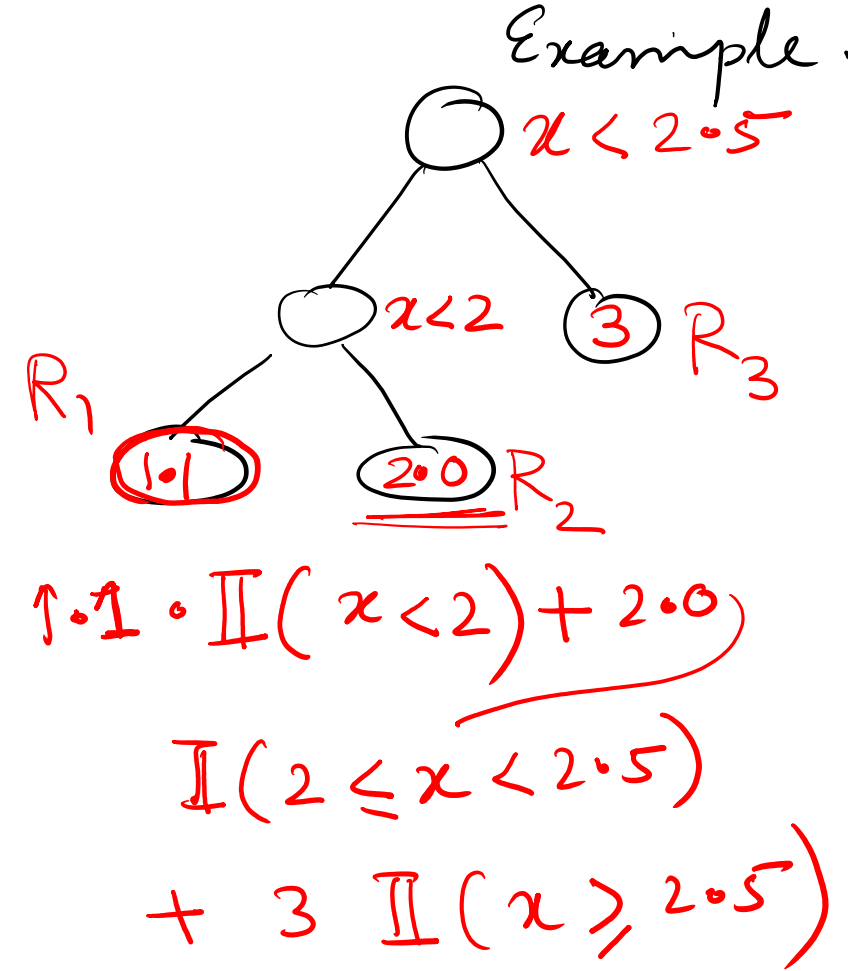
$$r_{im} = \frac{1}{2} \frac{\partial (y_i - f_i)^2}{\partial f_i} \bigg|_{f_i = f_{m-1}(x^i)} = - (y_i - f_{m-1}(x^i))$$

Gradient boosted regression trees.

- Assume tree is of the form

$$F_m(x) = \sum_{j=1}^{J_m} w_{jm} \mathbb{I}(x \in R_{jm})$$

- At each m , find residuals build the tree to fit the residuals as labels.



- Then resolve for the weights of each leaf by

$$\hat{w}_{jm} = \operatorname{argmin}_w \sum_{x_i \in R_{jm}} \ell(y_i, f_{m-1}(x_i) + w)$$

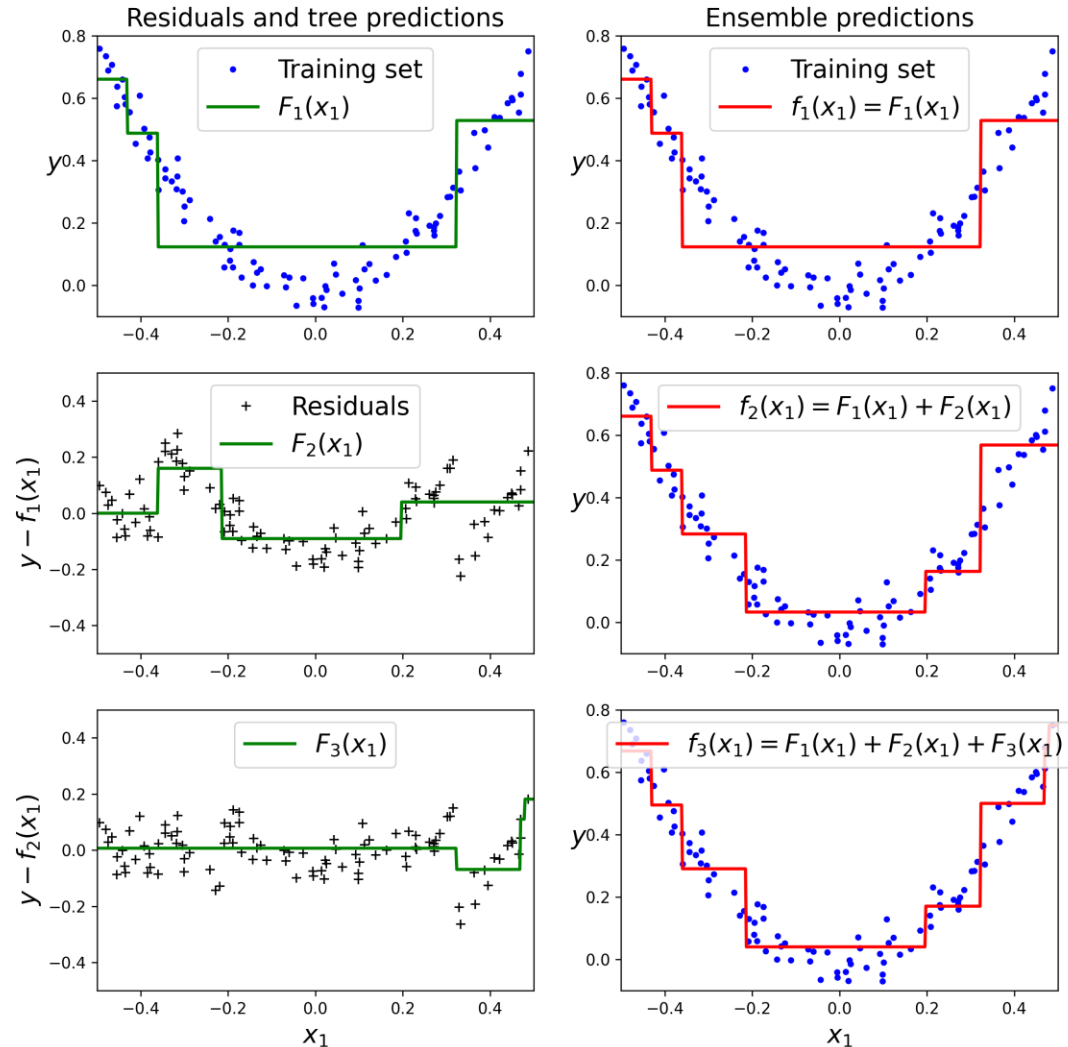
Boosted regression trees with square loss

- When loss function is square loss

- Residue is:

- Weight of each leaf

- <https://drive.google.com/file/d/1a7kfsrBtjwhP4x7XbwOtbWPG3oxdP5WS/view?usp=sharing>



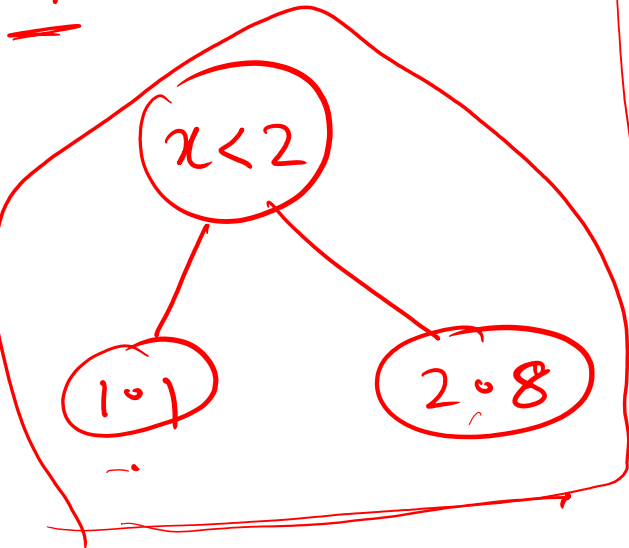
Example: Huber loss

$$L_{\delta}(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for } |y - f(x)| \leq \delta, \\ \delta \cdot (|y - f(x)| - \frac{1}{2}\delta), & \text{otherwise.} \end{cases}$$

$$\frac{\partial L}{\partial f} = \begin{cases} -(y - f(x)) & \text{for } |y - f(x)| \leq \delta \\ -\delta(\text{sign}(y - f(x))) & \text{otherwise} \end{cases}$$

$$\delta = 0.1$$

f_1



x	y	F_1	$\frac{\partial L}{\partial F}$
1	1	1.0	0.1
1.0	1.2	1.0	0.1
2	2.0	2.8	0.1
2.2	1.9	2.8	
4	3	2.8	
4.2	2.8	2.8	

Other variants

- Extreme gradient boosting (XGBoost).
- Significantly more evolved.
- But high accuracy with high speed training
- Extensively used